

01 — Frontend Requirements

Product: Endpoint IQ **Document:** Frontend Requirements **Source of truth:** *AI-Driven API Testing Platform — Executive Summary & Architecture Guide (v1.0, May 2026)*

1. Frontend Purpose

Endpoint IQ's frontend is the React/TypeScript web application through which a developer or QA engineer sets up an API testing project, imports an OpenAPI/Swagger specification, configures roles and credentials, launches automated test runs, monitors their execution, and reviews AI-generated analysis and a final report.

The frontend does not execute tests itself and does not run the AI analysis — those responsibilities belong to the Orchestrator, Workers, and AI Analysis Engine on the backend. The frontend's job is to **configure, trigger, observe, and explain**.

2. Frontend Scope

In scope: - Project creation and management - OpenAPI/Swagger import (by URL) - Endpoint discovery/viewing (read-only list derived from the imported spec) - Role configuration (Admin, Client, Company, etc. — project-defined) - Credential configuration per role - Triggering test runs - Real-time (or polling-based) monitoring of test execution progress - Viewing test results (pass/fail per endpoint per role) - Viewing failed API details (request, response, diffs, error info) - Viewing AI analysis (root cause, recommendations, missing coverage, security findings) - Downloading the final report (PDF/HTML, per the Report Service)

Out of scope (owned by other services, per the architecture): - Actually executing HTTP calls against the target API (Workers) - Parsing the OpenAPI spec (Spec Import Service) - Generating the PDF/HTML report file (Report Service) - Running the LLM analysis (AI Analysis Engine) - Managing JWTs for the *target* API under test (Auth Service / Session Store) — the frontend never sees these tokens; they are stored server-side in Redis as HttpOnly session data.

3. Target Users

- **Developers** who own an API and want automated regression/role-coverage testing.
- **QA engineers** validating role-based access and endpoint correctness.
- **Engineering leads** who review AI-flagged issues and download reports for stakeholders.

The architecture document does not define distinct frontend-facing user permission tiers (e.g., admin vs. viewer of the Endpoint IQ platform itself). This is marked as:

Open Decision: Whether Endpoint IQ itself has multiple platform-user roles (e.g., project owner vs. read-only viewer) is not defined in the source document. MVP assumes a single authenticated platform-user type with full access to their own projects.

4. User Workflow

The frontend must support this end-to-end flow, directly mirroring the architecture's "How Data Flows Step by Step" section:

flowchart LR

A[Project Setup] --> B[Import OpenAPI/Swagger]

```

B --> C[Endpoint Discovery]
C --> D[Role Configuration]
D --> E[Credential Configuration]
E --> F[Start Test Run]
F --> G[Monitor Test Execution]
G --> H[View Test Results]
H --> I[View Failed API Details]
I --> J[View AI Analysis]
J --> K[View Recommendations / Fix Info]
K --> L[Download Final Report]

```

5. Functional Requirements

ID	Requirement	Source
FR-1	User can create a new project with a name and Swagger/OpenAPI URL.	§3.1 Step 1
FR-2	Frontend triggers spec import via a backend call and displays the resulting endpoint list.	§3.1, §4.4
FR-3	User can view all discovered endpoints (method, path, schema info).	§4.1
FR-4	User can define roles for a project (e.g. Admin, Client, Company).	§5 Data Model (roles table)
FR-5	User can enter credentials per role (username/password); frontend never stores or displays raw credentials after submission — they are sent once and encrypted server-side.	§5, §6 Security
FR-6	User can start a test run for a project.	§3.3
FR-7	Frontend displays live/near-live progress of a running test (jobs completed vs. total).	§4.5 Orchestrator
FR-8	User can view a results dashboard: pass/fail summary per endpoint and per role.	§4.9 Report Service
FR-9	User can drill into a failed endpoint to see request, response, headers, status code, response time, and expected-vs-actual.	§4.9

ID	Requirement	Source
FR-10	User can view AI analysis: root cause, recommended fix, missing coverage, security findings — clearly labeled as AI-generated.	§4.8, Product Behavior notes
FR-11	User can download the final report file.	§4.9
FR-12	Frontend surfaces errors from the target API under test <i>and</i> errors from the Endpoint IQ backend distinctly.	§6 Security, §10 Risks
FR-13	Frontend never stores target-API JWTs in localStorage; it relies on backend-managed HttpOnly session cookies.	§4.3, §6

6. Non-Functional Requirements

Category	Requirement
Performance	Endpoint lists and result tables must remain usable for large APIs (100+ endpoints); use pagination/virtualization where needed.
Reliability	Frontend must clearly reflect backend job/queue state (pending, running, retried, failed) without silently freezing during long-running test runs.
Security	No sensitive tokens or credentials in localStorage/sessionStorage. All requests over HTTPS. Inputs validated client-side as a first line of defense (server remains authoritative).
Accessibility	Keyboard-navigable forms and tables; sufficient color contrast for status indicators (see 03-UI-UX-Specification.md).
Observability	Frontend errors should be catchable/loggable (see 08-Frontend-Development-Guidelines.md).
Maintainability	Feature-based structure so AI coding agents and developers can extend one workflow step without touching unrelated code (see 02-Frontend-Architecture.md).

7. MVP Functionality

Per the source document's Product Roadmap (§13, "MVP (v0)": *basic UI, single local LLM, happy-path tests, simple report, internal beta*), the MVP frontend must support:

- Project creation + Swagger URL import
- Endpoint viewing

- Role + credential configuration
- Starting a test run (happy-path tests)
- Basic progress monitoring
- Results dashboard (pass/fail)
- Failed API detail view
- AI analysis view (single local LLM output)
- Simple report download

8. Future / Optional Functionality

Per the Roadmap's v1/v2 phases:

- Multi-role support refinements and LLM fallback indicators (v1 — cloud LLM fallback surfaced in UI, e.g. “analyzed via Claude” badge)
- RBAC + audit log views (v1)
- Performance-testing result views (v2)
- Security fuzzing agent results (v2)
- Predictive test generation UI (v2)
- Email/Slack notification configuration (mentioned as “Optional” in §4.9 Report Service)

Open Decision: Notification preferences (email/Slack) UI is not detailed in the source beyond being “optional.” Treat as post-MVP.

9. User Actions

- Create / view / delete a project (*delete not explicitly defined — see Open Decision below*)
- Submit a Swagger URL for import
- View endpoint list
- Add/edit/remove a role
- Add/edit credentials for a role
- Start a test run
- Cancel a running test run (*not explicitly defined — see Open Decision below*)
- View live progress
- Filter/sort test results
- Open a failed endpoint's detail view
- Read AI analysis and recommended fix
- Download the report

Open Decision: Project/run deletion, run cancellation, and re-running a previous test run are not explicitly defined in the source architecture. Recommend exposing these as standard CRUD/control actions against the projects and test_runs tables, but flag to backend team for explicit API contract (see 05-Frontend-API-Integration.md, “Proposed API Contract” entries).

10. Acceptance Criteria (MVP)

- ☐ A user can go from “no project” to “downloaded report” using only the UI, with no manual API calls.
- ☐ Every step in the workflow diagram (§4) has a corresponding page/route (see 04-Frontend-Pages-and-Routes.md).
- ☐ Endpoint list accurately reflects the imported OpenAPI spec.
- ☐ Test run progress updates without requiring a manual page refresh.
- ☐ Failed endpoints show request/response/headers/status/response time/diff.
- ☐ AI analysis is visually distinct from raw test results and labeled as AI-generated.

- ☐ Report can be downloaded as a file from the browser.
- ☐ No JWT or credential value is ever written to localStorage/sessionStorage.

Cross-references: 04-Frontend-Pages-and-Routes.md (page-by-page detail), 05-Frontend-API-Integration.md (API contracts), 07-Frontend-Test-Results-and-AI-Report-UI.md (results/AI UI detail).